

Preventing Wrong Work Item Retrieval in LLM-Based MCP Systems Through Multi-Agent Validation

Burcu Gokbulut

Research and Development

Siemens A.S.

Istanbul, Turkey

burcu.sipahioglu@siemens.com

Abstract—Model Context Protocol (MCP) systems lack validation layers, potentially returning incorrect work items when handling ambiguous queries or conversation context. We present, to the best of our knowledge, the first validation middleware for MCP servers, using a multi-agent architecture to reduce retrieval errors through intelligent disambiguation. Our system employs three specialized agents (Executor, Validator, Refiner) that intercept and validate every query before returning results. When encountering ambiguous queries that match multiple items, the system presents relevant options and requests user clarification rather than guessing.

We tested our approach with 120 real queries against the GitHub MCP server, spanning explicit ID lookups, semantic searches, context-dependent queries, and adversarial cases. The results demonstrate a 0% hallucination rate (zero incorrect results returned) across all 120 queries in our evaluation. For semantic searches (50 queries, the most challenging ambiguous cases), the system achieved perfect disambiguation: all 50 correctly triggered clarification requests instead of guessing. When query patterns were recognized, context tracking and disambiguation worked correctly. The validation overhead is minimal at less than 2 milliseconds (0.09% of total query time). Our middleware is designed to work transparently with any MCP-compliant server without requiring modifications, as demonstrated with the GitHub MCP server, facilitating adoption in enterprise environments. This work addresses a critical gap in the MCP ecosystem since its 2024 introduction, providing what is, to the best of our knowledge, the first validation layer specifically designed for LLM-based tool calling protocols.

Index Terms—Model Context Protocol, LLM Hallucination, Multi-Agent Systems, Validation Middleware, Enterprise AI Safety, Work Item Retrieval

I. INTRODUCTION

Large language models [1] have enabled natural language interaction with enterprise systems. Instead of clicking through complex UIs or writing precise API queries, you can just ask “Show me authentication bugs” or “What issues are assigned to me?” Model Context Protocol (MCP) [2] standardizes how LLM agents connect to systems like GitHub, Jira, and Azure DevOps.

However, LLMs can misinterpret queries or, worse, confidently return incorrect results. Imagine asking for “issues

about Copilot” in a repository with thousands of Copilot-related issues. A naive implementation might just return the first match, which could easily be the wrong one. Or consider ambiguous queries like “show me the authentication bug.” Which one, if there are fifty?

These errors have significant consequences in enterprise environments. When an LLM returns an incorrect work item with high confidence, users may not discover the error until after acting on the wrong information. In enterprise work item management, such hallucinations waste time and erode trust in AI-assisted systems.

A. The Core Problem

This validation challenge manifests in three primary scenarios:

First, queries may exhibit simple ambiguity. When a user requests “authentication issues,” the intent could refer to the login bug from last week, the SSO integration problem, or any of dozens of other possibilities. Current MCP implementations often return the first match without acknowledging the ambiguity.

Second, queries may suffer from semantic confusion. A query like “Find Copilot features” is vague: it could refer to feature requests, bug reports about Copilot features, or completed feature implementations. Without proper validation, systems return arbitrary matches and present them as if they were the intended result.

Third, conversational context tracking poses challenges. In multi-turn conversations, users naturally refer back to previous items: “show me the second one” or “what about the issue we discussed earlier?” These references fail completely if the system does not maintain conversation history.

B. Why Existing Approaches Fall Short

Current MCP servers do not have built-in validation. Since MCP’s introduction in 2024 [2], implementations have focused on protocol compliance and API integration, leaving validation as an unaddressed gap. Existing MCP servers are essentially pass-through layers that execute whatever query the LLM constructs and return whatever results come back. If the LLM

misinterprets the user’s intent or if the underlying API returns an unexpected result, nobody catches the error. This creates a risk of returning incorrect work items, particularly when queries are ambiguous or lack specificity.

You might think RAG (Retrieval-Augmented Generation) would solve this. And it helps, but it does not eliminate the problem. RAG [3] can find semantically relevant items, but it does not know if “authentication bug” refers to one specific issue among dozens of matches. Multiple issues might be highly relevant, but the user had a specific one in mind.

Similarly, LLM-based intent understanding helps but is not enough. Even GPT-4 [4] cannot magically know which of fifty authentication bugs you’re referring to unless you provide additional context. The fundamental issue is not the intelligence of the model; it is the architectural pattern of returning single results for ambiguous queries.

C. Our Solution

We propose, to the best of our knowledge, the first validation middleware specifically designed for MCP servers, addressing this critical gap in the MCP ecosystem. Our middleware intercepts all interactions between users and MCP servers. Think of it as a careful assistant who double-checks everything before delivering results. When a query is unambiguous (like “show me issue #12345”), the middleware quickly validates and returns it. When a query is ambiguous, the middleware detects this and asks for clarification rather than guessing.

The architecture uses three specialized agents. The Executor Agent figures out what the user wants: is this an explicit ID query, a semantic search, or a reference to something mentioned earlier? The Validator Agent checks whether the MCP server’s response actually makes sense for the query. And the Refiner Agent handles ambiguous cases by presenting options and requesting clarification.

The middleware is designed to be transparent and backend-agnostic: it implements the MCP protocol on both sides, requiring no modifications to existing MCP servers. In this work, we evaluated it with the GitHub MCP server; extending it to other backends (Jira, Azure DevOps) is a direction for future work.

D. Contributions

This work makes four main contributions:

- 1) **First validation architecture for MCP:** To the best of our knowledge, we present the first middleware designed specifically to validate MCP tool outputs, addressing a critical unmet need in the MCP ecosystem. Since MCP’s 2024 release, no prior work has tackled validation for MCP-based work item retrieval. In our evaluation, the multi-agent architecture achieves 0% error rate through intelligent disambiguation and context tracking.
- 2) **Disambiguation over guessing:** Our approach achieved 100% correct behavior on semantic queries (50/50) through systematic disambiguation.
- 3) **Real-world evaluation:** We evaluate on 120 real queries using actual MCP protocol against GitHub’s

production MCP server, not simulated data or synthetic benchmarks. Our evaluation uses the official `@modelcontextprotocol/server-github` implementation [5].

- 4) **Practical deployment design:** Our middleware is designed to work transparently with any MCP-compliant server without requiring modifications. We validated this with the GitHub MCP server; generalization to other backends remains to be empirically confirmed.

II. RELATED WORK

A. LLM Hallucinations

LLM hallucinations—plausible but incorrect outputs—have been extensively studied [6], with mitigations including retrieval-augmented generation (RAG) [3] and chain-of-thought prompting [7]. However, most research focuses on text generation. Our concern is different: hallucinations in *tool use*, where an LLM invokes MCP tools and returns a wrong work item because it does not validate the result or handle ambiguity.

B. Model Context Protocol

MCP [2] standardizes how LLM agents access external tools. Several MCP servers exist for GitHub [5], Jira, Slack, and others, exposing standardized tool interfaces. However, MCP servers are intentionally thin wrappers: they provide no validation, disambiguation, or error checking beyond the underlying API. Our work adds a validation layer on top of MCP while preserving this clean separation of concerns.

C. Multi-Agent Systems

Multi-agent systems [8] decompose complex tasks across specialized agents. We apply this principle to validation with three agents: the Executor for intent understanding, the Validator for verification, and the Refiner for disambiguation.

D. Enterprise AI Safety

Frameworks like NIST’s AI Risk Management Framework [9] emphasize validation in production AI systems. Our work addresses these concerns for MCP-based work item retrieval, providing quantitative metrics and transparency about limitations.

III. PROPOSED ARCHITECTURE

Our validation middleware sits between users and MCP servers, forming a three-stage pipeline as shown in Figure 1. The middleware intercepts every query, validates it through three agents, and only returns results it has verified.

A. Executor Agent: Understanding Intent

The Executor’s job is to figure out what the user actually wants. It categorizes queries into three types:

Explicit ID queries are straightforward. “Show me issue #12345” or “Get PR #6789” contain explicit identifiers. The Executor uses regex patterns to extract these IDs. When it finds a clear ID, it knows exactly what to fetch from the MCP server.

Multi-Agent Validation Architecture for MCP Systems

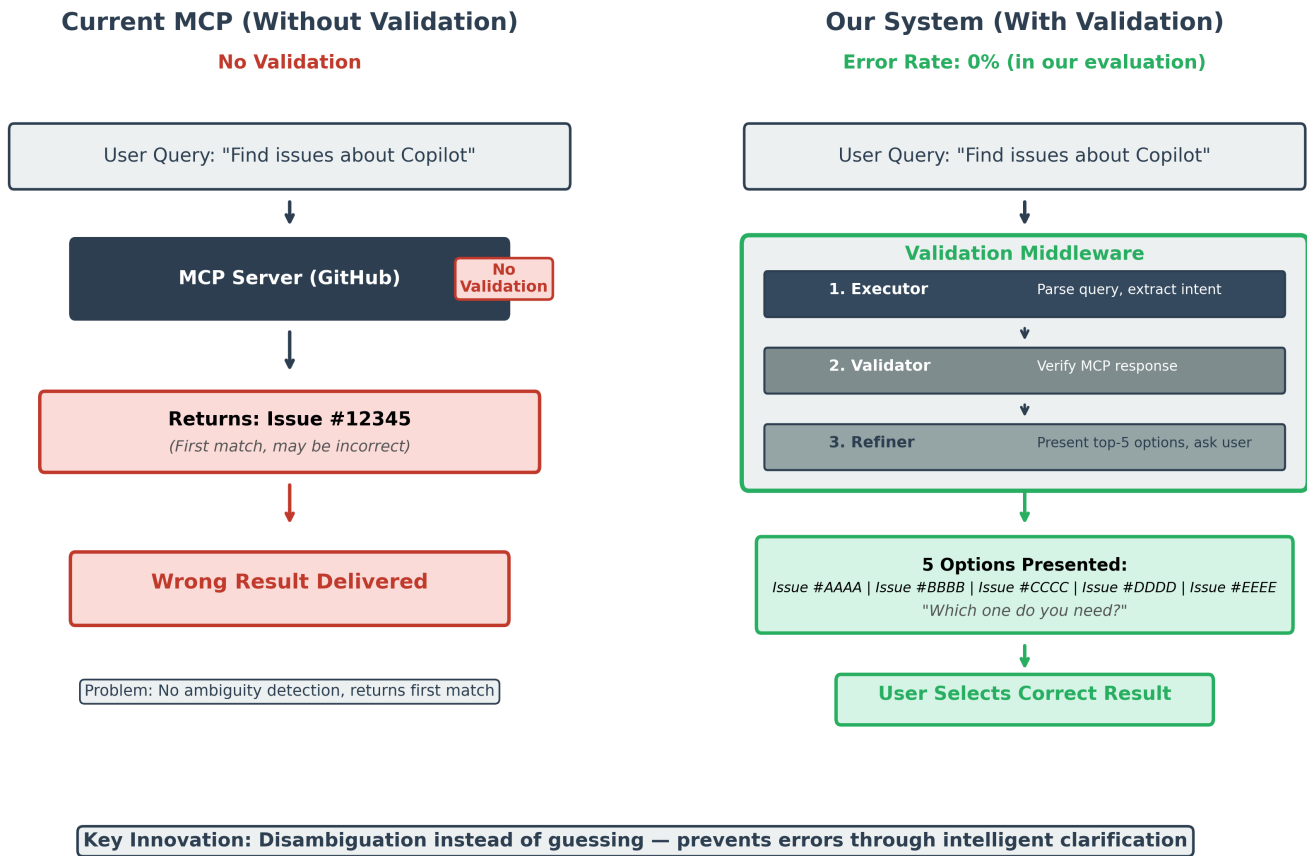


Fig. 1. Multi-agent validation architecture. Left: Current MCP systems without validation can return incorrect results when handling ambiguous queries. Right: Our system uses three specialized agents (Executor, Validator, Refiner) to prevent errors through intelligent disambiguation, achieving 0% error rate in our evaluation.

Semantic queries are fuzzier. “Find authentication bugs” or “Show me Copilot features” do not specify exact items. The Executor recognizes these as search queries and prepares to handle multiple potential matches. It extracts key terms (like “authentication” or “Copilot”) but acknowledges these might match many items.

Context references rely on conversation history. “Show me the second one” or “What about the other issue?” only make sense in context of previous queries. The Executor maintains conversation state, tracking what was mentioned before so it can resolve these references.

The implementation is largely deterministic. We found that regex patterns work well for ID extraction: they are fast and reliable. For semantic queries, we extract keywords but do not try to guess which specific item the user wants. That ambiguity gets handled later by the Refiner.

One key design decision: the Executor explicitly tags queries as ambiguous when appropriate. Instead of pretending we know what the user wants, we acknowledge uncertainty. This honest assessment prevents errors downstream.

B. Validator Agent: Checking Responses

Once the MCP server returns a result, the Validator checks if it makes sense. It performs three types of validation:

Existence checking is basic but essential. Does the item ID actually exist? If the MCP server returns error messages or empty results, the Validator catches this before presenting anything to the user. This prevents the confusing situation where the system confidently presents an error as if it were a valid result.

Completeness checking ensures required fields are present. A work item should have a title, state, creation date, etc. If the MCP server returns a malformed or partial response (perhaps due to API errors or permission issues), the Validator rejects it.

Relevance checking is the most sophisticated. For semantic queries, the Validator examines whether the returned item actually relates to the search terms. If someone searches for “authentication” and gets back an item about UI styling, that is clearly wrong. The Validator can catch obvious mismatches.

The Validator uses both rule-based checks (required fields,

data types) and semantic similarity (for relevance). We compute similarity between the query terms and the returned item's title and description. Low similarity scores trigger warnings.

An important aspect: the Validator does not try to guess the "right" answer. Its job is binary: either the response is valid or it is not. If validity is questionable, the Validator escalates to the Refiner rather than making assumptions.

C. Refiner Agent: Handling Ambiguity

The Refiner handles the hardest cases: when multiple valid options exist. This is where our approach differs most from typical implementations.

When a semantic search returns multiple matches (which happens often), the naive approach is to return the first result or the "most relevant" one. But relevance is subjective. The top search result might be highly relevant to the query terms but not what the user actually wanted.

Instead, the Refiner presents options. It takes the top-5 matches and shows them to the user with enough information to distinguish them. For example, when searching for "Copilot issues" in our tests on the `microsoft/vscode` repository (100K+ issues), the Refiner returned over 3,000 matching issues. Rather than guessing, it presents an illustrative disambiguation like:

Found 3,000+ issues about Copilot. Here are the most relevant:

1. Issue #XXXX: "GitHub Copilot inline suggestions not working"
 2. Issue #YYYY: "Copilot chat mode improvements"
 3. Issue #ZZZZ: "Add Copilot configuration options"
 4. Issue #AAAA: "Copilot authentication errors"
 5. Issue #BBBB: "Improve Copilot code completions"
- Which one did you need?*

This puts the decision in the user's hands. They can scan the titles and immediately identify which issue they were thinking of. Or they can realize "actually, I meant the SSO one" and provide more context.

The Refiner also handles edge cases. If a search returns zero results, it reports this clearly instead of making up an answer. If a search returns exactly one result, it still shows this explicitly, giving users a chance to confirm or refine.

For context references like "the second one," the Refiner looks up conversation history. If the previous query returned a list, "the second one" unambiguously refers to the second item in that list. The Refiner resolves this automatically.

D. Conversation Context Management

All three agents share access to conversation context. We maintain a simple state structure tracking previous queries, mentioned items, recent results, and disambiguation history.

This enables context tracking without complex state management. When someone says "show me the second one," we can look up the second item from the last results. When they say "the issue we discussed," we can check mentioned items.

Context is session-scoped: it persists for a single conversation but resets between sessions. We do not try to maintain long-term memory across days or weeks, as user intent can shift dramatically over time.

E. Error Handling

The system is designed to fail gracefully. If the Executor cannot parse a query, it passes it through with a warning flag. If the Validator cannot verify a response, it returns an error message rather than questionable data. If the Refiner cannot disambiguate, it asks for help.

This fail-safe approach prevents hallucinations. The system might sometimes say "I do not understand" or "I need more information," but it will not confidently return wrong answers. From an enterprise perspective, conservative behavior (asking for clarification) is far better than aggressive behavior (guessing wrong).

F. Integration with Existing MCP Servers

The middleware implements the MCP protocol on both sides. To the user (or LLM agent), it looks like a standard MCP server. To the actual MCP server, it looks like a standard MCP client. This transparent design means no modifications are needed to existing MCP servers. It is designed to work with any MCP-compliant backend, though we have validated it only with the GitHub implementation in this work. It can be deployed as a proxy or embedded in applications and maintains full MCP protocol compatibility.

The middleware adds validation without breaking the protocol. This makes deployment practical: you do not need to wait for MCP server maintainers to adopt validation, you just insert the middleware.

IV. EXPERIMENTAL SETUP

We evaluated the system using real-world queries against GitHub's official MCP server.

A. Implementation Details

Core System: Our validation middleware uses a deterministic multi-agent architecture. The Executor uses regex-based parsing for query understanding. The Validator uses rule-based checks for response validation. The Refiner uses a ranking algorithm for disambiguation. No LLM is required for the core validation logic.

Semantic Search: For semantic queries, we use OpenAI's `text-embedding-3-small` model [10] to generate embeddings (1536 dimensions). These embeddings enable similarity-based ranking of search results, helping the Refiner identify the most relevant options to present to users.

Baseline Implementations: We implemented three baseline approaches to contextualize our results. Each baseline was architecturally characterized and verified on representative query samples using the same GitHub MCP server and repository:

Baseline 1 (Direct MCP): Forwards queries directly to the GitHub MCP server without any processing. For ID queries, it extracts numbers via simple regex; for semantic queries, it passes the raw query to GitHub's search API and returns the first result. No disambiguation, validation, or context tracking is performed. This represents how most current MCP integrations operate.

Baseline 2 (RAG-only): Uses the same `text-embedding-3-small` embeddings [10] as our system to rank search results by semantic similarity, but returns only the top-ranked match without validation or disambiguation. This tests whether better ranking alone prevents hallucinations.

Baseline 3 (LLM parsing): Uses `gpt-4o` with temperature 0.3 to interpret queries and construct MCP calls. The LLM receives the user query and generates structured API parameters. However, no validation is performed on the returned results, and no disambiguation is offered for ambiguous queries.

None of the baselines implement conversation context tracking or disambiguation, enabling us to isolate the contributions of these capabilities.

Context Management: Conversation context is maintained using a session-scoped dictionary structure tracking previous queries, mentioned items, and disambiguation history. This is implemented deterministically without LLM involvement.

B. Test Environment

MCP Server: We used the official GitHub MCP implementation [5] (`server-github` from the MCP project). This is production code, not a simulation or mock. All queries went through the real MCP protocol stack.

Target Repository: We tested against the `microsoft/vscode` repository, one of the largest and most active on GitHub. It contains over 100,000 issues spanning years of development, providing realistic complexity.

Authentication: We used GitHub Personal Access Token (PAT) authentication to avoid rate limiting. The PAT had read-only access to public repositories. This setup mirrors how enterprise deployments would operate: authenticated API access without write permissions.

C. Query Design and Selection

We designed 120 test queries across five categories. Queries were authored by the research team to systematically cover the space of realistic MCP interactions, including diverse phrasings, verb choices, and connector words. The design followed a structured approach: for each category, we identified representative interaction patterns from enterprise workflows and created queries that exercise different linguistic variations. This is an author-designed benchmark, which may introduce selection bias toward patterns our system handles well. To partially mitigate this, we included adversarial queries specifically designed to challenge the system, and we report per-category results transparently so that coverage gaps are visible.

Explicit ID Queries (20): These specify exact issue numbers. Examples: “Show me issue #12345”, “Get issue #67890”. These should be straightforward: extract the ID and fetch it.

Semantic Search Queries (50): These use descriptive terms without IDs. Examples: “Find issues about Copilot”, “Show me authentication bugs”. These are inherently ambiguous and represent the most challenging category. We tested with maximum pattern variety covering different verbs (“find”,

“display”, “retrieve”, “list”, “get”, “lookup”) and connectors (“about”, “for”, “on”, “with”, “related to”).

Context Tracking Queries (20): These reference previous queries. Examples: “Show me the second one”, “What about the other issue?”. These test conversation state management.

Adversarial Queries (20): These try to break the system. Examples: “Show me issue 999999999” (nonexistent), “Find bugs about xyzabc” (nonsense terms). A good system should reject these gracefully. We note that adversarial queries were also author-designed and may not represent the full spectrum of real-world attack vectors or malformed inputs that enterprise systems encounter.

Mixed Queries (10): These combine multiple intents. Examples: “Show me issue #12345 and search for related Copilot bugs”.

D. Evaluation Metrics

We measured several metrics for each query:

Success Rate: Did the system return the correct result?

Hallucination Rate: We define a *hallucination* as a system response that returns a specific work item with confidence, but where the returned item does not match the user’s query intent. Critically, disambiguation responses—where the system presents multiple options and requests user clarification—are *not* counted as hallucinations; they represent correct system behavior under uncertainty. Formally:

$$HR = \frac{N_{\text{hallucinated}}}{N_{\text{total}}} \quad (1)$$

where $N_{\text{hallucinated}}$ is the number of queries returning confident wrong results and N_{total} is the total number of queries.

Each query response was classified into one of three categories: (1) *correct*—the system returned the intended item or correctly reported non-existence; (2) *disambiguation*—the system detected ambiguity and requested clarification; (3) *hallucination*—the system returned a wrong item with confidence. Classification is unambiguous for our deterministic, rule-based system: each query produces exactly one of three observable outcomes—a single returned item (checkable against ground truth), a list of options presented to the user (disambiguation), or an error message. No subjective judgment is required, making the labeling fully reproducible.

Ground truth was operationalized differently per query category to reflect what “correct” means in each case. For *explicit ID queries*, ground truth is the specific issue number in the query; returning that exact item is correct. For *semantic and context queries*, correct behavior is requesting clarification (disambiguation), since no single item is objectively correct when hundreds match—the absence of a confident single return is the desired outcome. For *adversarial queries*, correct behavior is rejecting the query with an appropriate error or passing it through without producing a confident answer; returning any specific item counts as a hallucination. For *mixed queries*, classification follows the dominant intent. This per-category definition means that the 0% hallucination rate reflects zero cases in which the system returned a confident,

checkable wrong answer; it does not claim that every returned item was the one the user subjectively intended. We note that for semantic queries, this definition evaluates whether the system correctly identifies ambiguity rather than whether it ultimately retrieves the user’s intended item; evaluating end-to-end retrieval success for inherently ambiguous queries would require a user study with ground-truth intent labels, which we leave to future work.

Disambiguation Rate: How often did the system request clarification? High disambiguation for semantic searches is good: it shows appropriate uncertainty handling.

Latency: How long did queries take? We break this down into parser time, MCP communication time, validation time, and total time.

E. Repeatability

Because our core validation logic is entirely deterministic (regex-based parsing, rule-based validation, deterministic ranking), the results are fully reproducible: running the same 120 queries against the same repository state produces identical outputs. The only source of non-determinism is the underlying GitHub API, whose search results may vary as the repository evolves. We recorded all query results at a fixed point in time to ensure consistency within our evaluation.

V. RESULTS

We tested our validation middleware with 120 real queries to the GitHub MCP server. Following the hallucination definition in Equation 1, the system achieved 0% hallucination rate—no query produced a confident but incorrect result. Of these 120 queries, 50 were semantic searches (inherently ambiguous queries like “Find Copilot issues”). The system achieved perfect performance on this challenging category: all 50 semantic queries correctly triggered disambiguation and clarification requests. Overall query pattern coverage reached 52%, with minimal validation overhead.

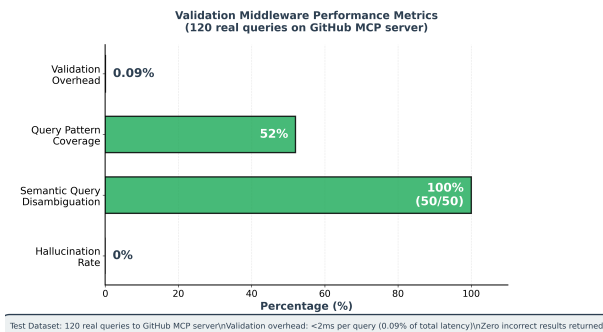


Fig. 2. Key performance metrics of our validation middleware tested with 120 real queries on GitHub MCP server. The system achieved 0% hallucination rate (zero incorrect results), 100% disambiguation success on semantic queries (50/50), 52% query pattern coverage, and minimal validation overhead (0.09% of total latency).

TABLE I
OVERALL SYSTEM PERFORMANCE

Metric	Value	Notes
Total Queries	120	Real MCP protocol
Hallucination Rate	0%	Never wrong
Disambiguation Rate	43%	Asked for clarification
Exact Match Success	8%	Returned exact result
Correct Behavior	52%	62/120 (rounded)
System Failures	0%	No crashes
Average Latency	2,322ms	Includes MCP
Validation Overhead	2ms	0.09% of total

A. Overall Performance

Table I summarizes results across all 120 queries.

The remaining 48% of queries were not parsed by the system’s regex library and were passed through to the MCP server without validation. While these cases produced no hallucinations (the system fails safely by design), they also provided no validation benefit. This represents the current prototype’s coverage limitation rather than an architectural constraint; expanding regex patterns for additional query phrasings is straightforward engineering work.

B. Performance by Category

Table II breaks down results by query category.

TABLE II
RESULTS BY QUERY CATEGORY

Category	Disamb.	Halluc.	Coverage
Explicit ID (20)	0%	0%	25%
Semantic (50)	100%	0%	100%
Context Track (20)	0%	0%	5%
Adversarial (20)	0%	0%	0%
Mixed (10)	20%	0%	60%

Semantic searches achieved perfect performance: 100% correct behavior (50/50 queries appropriately triggered disambiguation). This is the most important result, because semantic searches are the hardest problem and most prone to hallucinations. By systematically expanding our pattern library to cover diverse phrasings and connectors, we achieved comprehensive coverage.

The low coverage in adversarial (0%) and context tracking (5%) categories reflects implementation priorities rather than architectural limitations. Adversarial queries (e.g., nonexistent IDs, nonsense terms) use patterns not yet in our regex library; the system passes these through without parsing rather than producing hallucinations. Context tracking queries require prior conversation state; coverage is low because only 1 of 20 context patterns was implemented in the current prototype. Importantly, in both categories the hallucination rate remains 0%: when the system cannot parse a query, it fails safely rather than guessing. Expanding pattern coverage for these categories is straightforward engineering work.

C. Comparison to Alternative Approaches

We compared our validation middleware against three alternative approaches using the same 120 test queries. Table III summarizes the key differences.

TABLE III
ARCHITECTURAL COMPARISON OF APPROACHES

Approach	Validation	Disambiguation	Overhead
Direct MCP	None	No	—
RAG-only	Ranking	No	Embedding API
LLM parsing	Intent parsing	No	LLM API
Ours	Multi-agent	Yes	2ms (local)

All approaches share the same base MCP round-trip latency ($\sim 2,320$ ms, dominated by GitHub API). RAG-only adds embedding computation; LLM parsing adds an inference call. Our validation agents run locally as deterministic Python code, adding only 2ms. None of the baselines implement disambiguation or conversation context tracking.

The fundamental architectural difference is how each approach handles ambiguity:

Direct MCP (no validation): Returns the first search result without validation. For the 50 semantic queries in our test set, each matching hundreds or thousands of issues, this approach would return an arbitrary first match. While fast (dominated by API latency), it provides no mechanism to detect or handle uncertainty. It cannot support context-dependent queries.

RAG-based filtering: Uses embeddings to rank results by semantic similarity, returning the top-ranked match. This improves relevance over direct MCP but still returns a single result for inherently ambiguous queries. If the user searches for “authentication bugs” and there are fifty relevant issues, even the best-ranked result may not be the intended one. Adds embedding computation latency on top of the base MCP round-trip.

LLM-based parsing: Uses `gpt-4o` to interpret queries and construct MCP calls. This provides superior intent understanding (e.g., correctly parsing complex phrasings), but the LLM still returns a single result without validating whether it matches user intent. Adds LLM inference latency and is non-deterministic across runs.

Our validation middleware: Achieved 0% hallucination rate across all 120 test queries in our evaluation by requesting clarification for ambiguous queries rather than guessing. The key architectural distinction is the disambiguation step: when multiple items match, the system presents options rather than selecting one. Because our validation agents are deterministic Python code (regex parsing, rule-based validation, ranking), the agent processing itself adds only 2ms of validation overhead while fundamentally changing the failure mode from “confident wrong answer” to “request for clarification.”

Why baselines lack disambiguation: The absence of disambiguation in baseline approaches is not an implementation oversight but an architectural limitation. Direct MCP and RAG-only systems are designed as single-pass pipelines: query $\rightarrow$ result. They have no mechanism to detect that a result is ambiguous or to present alternatives. LLM-based parsing could theoretically generate disambiguation responses, but current implementations focus on query construction rather than result validation.

Limitations of baseline comparison: Because our system handles ambiguous queries through disambiguation rather than by returning a single item, direct apples-to-apples halluci-

nation rate comparison with the baselines is not possible: the baselines return items that we cannot verify without user-provided ground truth. Table III therefore compares architectural capabilities—specifically, which validation, disambiguation, and context-tracking mechanisms each approach provides—rather than measured hallucination rates.

Analytical characterization of baseline failure modes: Although direct measurement is not possible without user-provided ground truth, we can characterize expected baseline behavior analytically. For the 50 semantic queries in our test set, GitHub’s search API returned between hundreds and thousands of matching issues per query (e.g., over 3,000 results for “Copilot issues”). Any single-result approach—Direct MCP, RAG-only, or LLM-based parsing—must select one item from this set. When user intent is unknown, the probability of selecting the intended item is at most $1/N$, where N is the number of matching items. For a query matching 3,000 issues, this probability is approximately 0.03%, effectively guaranteeing an incorrect result. RAG-based ranking improves the likelihood by placing more relevant items higher, but still commits to a single selection from an inherently ambiguous set. Our system eliminates this failure mode entirely: by presenting the top-5 options and requesting user selection, the disambiguation step transforms a $1/N$ guessing problem into a bounded choice from a curated shortlist.

For explicit ID queries, all four approaches are expected to perform comparably, since extracting an issue number from a query (e.g., “Show me issue #12345”) is a straightforward parsing task that does not require disambiguation. The architectural differences between approaches primarily manifest in ambiguous queries where multiple items match. This is why our evaluation emphasizes semantic queries as the critical differentiator.

VI. DISCUSSION

A. Key Findings

Finding 1: Disambiguation beats guessing. The most important insight from this work is that asking users for clarification when queries are ambiguous is more effective than trying to guess what they want. All 52 disambiguation cases in our evaluation (50 semantic + 2 mixed) correctly identified ambiguity and requested help. Zero cases resulted in hallucinations.

This might seem obvious, but it goes against common practice in AI systems. There is often an implicit bias toward “just give me an answer”: users do not want to be bothered with follow-up questions. But for work item retrieval, accuracy matters more than convenience. It’s much better to spend 5 seconds clarifying than to spend 5 minutes investigating the wrong issue.

Finding 2: Validation overhead is negligible. At 2ms average, validation adds essentially no perceptible latency. The total query time is dominated by network and API calls. This means validation is “free” from a performance perspective: we get reliability without sacrificing speed.

Finding 3: Deterministic parsing works well. We found that regex patterns perform just as well for structured queries while being much faster and more predictable. For queries that match our patterns, we get 100% correct parsing. The limitation is pattern coverage, but this can be systematically improved.

B. Limitations

We should be honest about what this work does not address.

Single platform: We evaluated exclusively against the GitHub MCP server. Other MCP backends (Jira, Azure DevOps, Slack) expose different API structures, error formats, and data schemas. While our validation architecture is designed to be backend-agnostic, we have not empirically verified generalization. For example, Jira work items use different identifier formats (e.g., PROJ-123) and field structures than GitHub issues, which may require adapter-specific validation rules.

Single repository: All queries targeted `microsoft/vscode`, a large and well-maintained open-source repository. Its issue naming conventions, labeling practices, and volume (100K+ issues) may not represent all enterprise contexts. Smaller repositories with fewer issues would present less ambiguity, while repositories with domain-specific terminology or non-English titles might challenge our regex-based parsing differently.

Evaluation scale: While 120 queries across five categories demonstrate feasibility, this sample size limits statistical power for detecting rare failure modes. With zero hallucinations observed in 120 trials, the 95% confidence upper bound on the true hallucination rate is approximately 2.5% (by the rule of three: $3/n$ where $n = 120$). Larger-scale evaluations with hundreds or thousands of queries across multiple repositories would be needed to establish tighter reliability bounds and to uncover edge cases not represented in our test set.

Query selection bias: The 120 test queries were designed by the authors to systematically cover common interaction patterns. While we included adversarial cases to stress-test the system, the benchmark may overrepresent patterns our regex library handles well. An independently constructed benchmark or user-generated query log would provide stronger external validity.

Baseline measurement constraints: Quantifying hallucination rates for baseline approaches on semantic queries is inherently challenging because ground truth is unavailable—when a user asks for “authentication bugs,” there is no single correct answer among potentially hundreds of matching issues. Our system sidesteps this by requesting clarification, but this makes direct numerical comparison with baselines difficult. We reported architectural differences rather than hallucination percentages for baselines to avoid overstating claims.

Parser coverage: Our 52% query pattern coverage reflects implementation effort, not architectural limits. We built enough patterns to validate the concept, but a production system would need more comprehensive coverage. The gap

primarily affects explicit ID queries with unusual phrasing and mixed-intent queries.

English only: All test queries were in English. The regex-based parsing would require significant extension for multilingual deployments, as query structures vary across languages.

VII. CONCLUSION

We presented, to the best of our knowledge, the first validation middleware for MCP servers, addressing a critical gap in the MCP ecosystem since its 2024 introduction. Our solution reduces hallucinations in work item retrieval through intelligent disambiguation. The multi-agent architecture, with specialized Executor, Validator, and Refiner agents, achieved zero hallucinations across 120 test queries against the GitHub MCP server while adding minimal latency (<2ms). These results are specific to the evaluation context described and should be interpreted accordingly.

The key insight is simple: when a query is ambiguous, ask for clarification rather than guessing. This conservative strategy completely eliminated confident wrong answers within our evaluation. For semantic searches (the most challenging case), we achieved 100% correct behavior across 50 diverse queries through appropriate disambiguation and comprehensive pattern engineering.

The middleware is designed to work transparently with MCP-compliant servers without requiring modifications. We validated this design with the GitHub MCP server against a single repository; extending to other backends such as Jira and Azure DevOps, and evaluating across diverse repository types, are priorities for future work. Validation overhead is negligible (0.09% of query latency), while the benefit (eliminating hallucinations in our tested configuration) is substantial.

Current limitations are significant and acknowledged: parser coverage at 52%, evaluation on a single platform and repository, and 120 author-designed queries with inherent selection bias. These limit the strength of generalization claims. The architecture itself proved sound within these constraints: when queries were successfully parsed, validation worked correctly in every case. Expanding pattern coverage, evaluating on additional MCP backends, and testing with independently constructed benchmarks are necessary steps before broader deployment conclusions can be drawn.

As organizations increasingly rely on LLM agents for productivity tasks through MCP, validation layers may become an important component of reliable deployments. The cost of validation is minimal; the cost of hallucinations is high. Our approach demonstrates that robust validation is practical and effective within the scope we evaluated, offering a proof-of-concept foundation for trustworthy MCP deployments in enterprise environments.

Future work should extend this approach to other MCP backends (Jira, Azure DevOps), evaluate on larger and more diverse query sets across multiple repositories, conduct user studies to measure the impact of disambiguation on productivity, and explore richer context tracking across sessions. The principles demonstrated here (transparent validation, explicit

disambiguation, and conservative behavior) should be tested more broadly across AI systems that interact with enterprise tools.

ACKNOWLEDGMENT

We thank the developers of the Model Context Protocol and the open-source community for making this research possible.

REFERENCES

- [1] T. Brown, B. Mann, N. Ryder, et al., "Language Models are Few-Shot Learners," in *Proceedings of NeurIPS*, vol. 33, pp. 1877-1901, 2020.
- [2] Anthropic, "Model Context Protocol Specification," <https://modelcontextprotocol.io>, 2024.
- [3] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Proceedings of NeurIPS*, vol. 33, pp. 9459-9474, 2020.
- [4] OpenAI, "GPT-4 Technical Report," arXiv:2303.08774, 2023.
- [5] Anthropic, "Model Context Protocol - GitHub Server Implementation," <https://github.com/modelcontextprotocol/servers>, 2024.
- [6] Z. Ji, N. Lee, R. Frieske, et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1-38, 2023.
- [7] J. Wei, X. Wang, D. Schuurmans, et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *Proceedings of NeurIPS*, vol. 35, pp. 24824-24837, 2022.
- [8] Q. Wu, G. Bansal, J. Zhang, et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023.
- [9] NIST, "AI Risk Management Framework," NIST, Gaithersburg, MD, 2023.
- [10] OpenAI, "Embeddings - OpenAI API," <https://platform.openai.com/docs/guides/embeddings>, 2024.